CSA0604 – DESIGN AND ANALYSIS OF ALGORITHMS

LAB EXPERIMENTS -SESSION 3

NAME : Shree Ranjane K

REG NO: 192471052

1. Number of Ways to Move a Ball Out of the Grid

```
#include <stdio.h>

int m, n;

int countPaths(int i, int j, int N) {
    if (i < 0 || i >= m || j < 0 || j >= n)
        return 1;

    if (N == 0)
        return 0;

    return countPaths(i - 1, j, N - 1) +
           countPaths(i + 1, j, N - 1) +
           countPaths(i, j - 1, N - 1) +
           countPaths(i, j + 1, N - 1);
}

int main() {
    m = 2;
    n = 2;
    printf("Output: %d\n", countPaths(0, 0, 2));

    m = 1;
```

```
    n = 3;
    printf("Output: %d\n", countPaths(0, 1, 3));

    return 0;
}
```

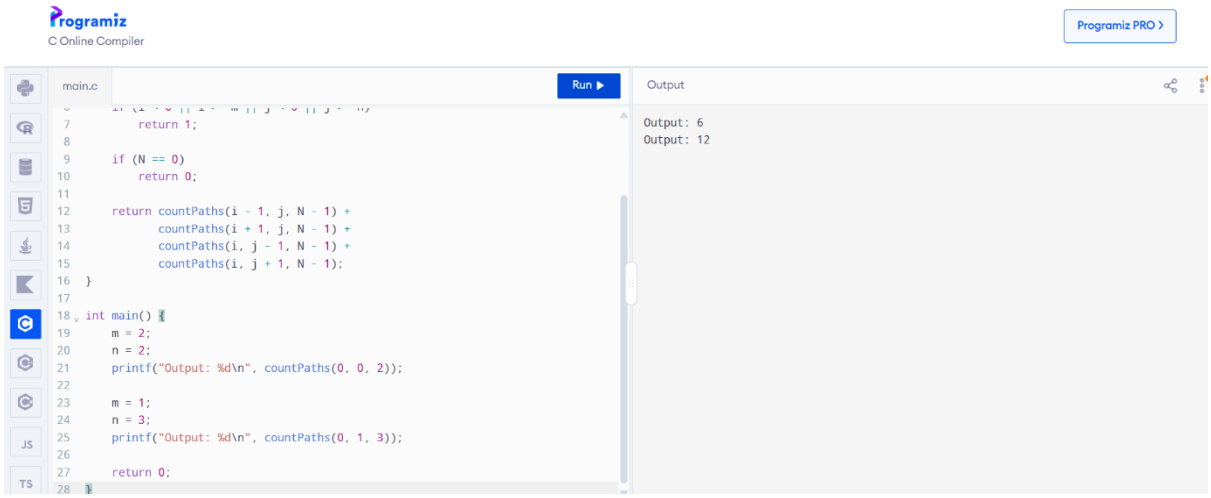
Output

Output: 6

Output: 12

Time Complexity: \(O(4^N)\)

Space Complexity: \(O(N)\) due to recursion.



2. House Robber II – Maximum Money Using Dynamic Programming

```
#include <stdio.h>

int robLinear(int nums[], int start, int end) {
    int prev1 = 0, prev2 = 0;

    for (int i = start; i <= end; i++) {
```

```
        int curr = nums[i] + prev2;
        if (prev1 > curr)
            curr = prev1;

        prev2 = prev1;
        prev1 = curr;
    }

    return prev1;
}

int rob(int nums[], int n) {
    if (n == 1)
        return nums[0];

    int a = robLinear(nums, 0, n - 2);
    int b = robLinear(nums, 1, n - 1);

    return (a > b) ? a : b;
}

int main() {
    int nums1[] = {2, 3, 2};
    int nums2[] = {1, 2, 3, 1};

    printf("Maximum money: %d\n", rob(nums1, 3));
    printf("Maximum money: %d\n", rob(nums2, 4));

    return 0;
```

}

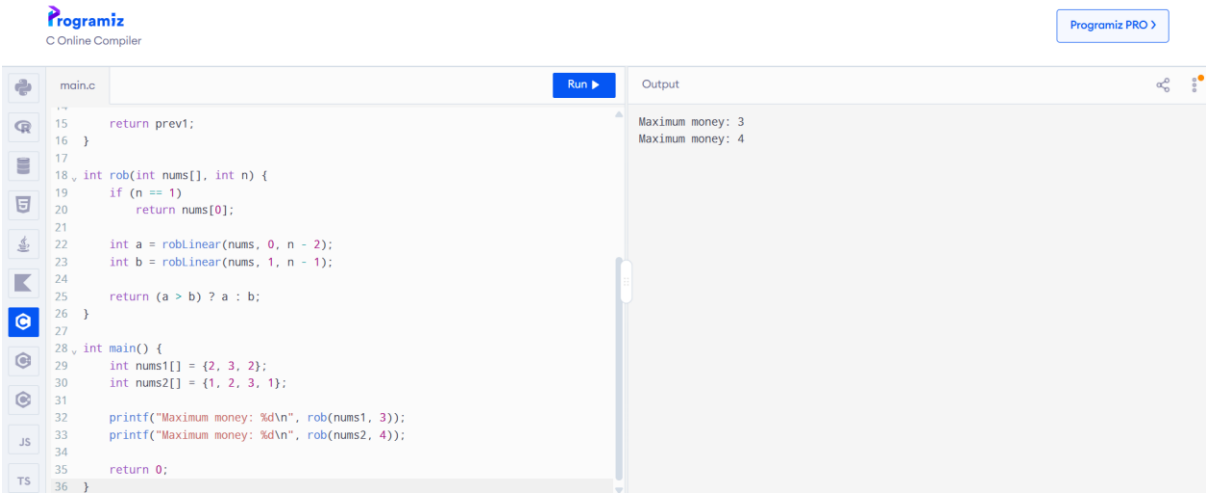
Output

Maximum money: 3

Maximum money: 4

Time Complexity: \(O(n)\)

Space Complexity: \(O(1)\)



3. Climbing Stairs – Number of Distinct Ways Using Dynamic Programming

```
#include <stdio.h>

int climbStairs(int n) {
    if (n <= 2)
        return n;

    int a = 1, b = 2, c;

    for (int i = 3; i <= n; i++) {
        c = a + b;
        a = b;
```

```
        b = c;
    }

    return b;
}

int main() {
    printf("Output: %d\n", climbStairs(4));
    printf("Output: %d\n", climbStairs(3));

    return 0;
}
```

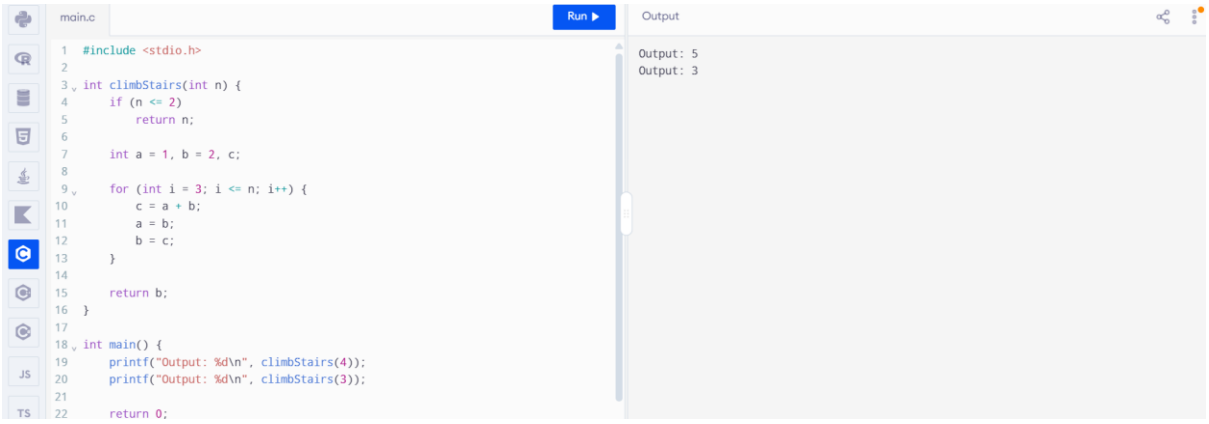
Output

Output: 5

Output: 3

Time Complexity: \(O(n)\)

Space Complexity: \(O(1)\)



4. Unique Paths in a Grid Using Dynamic Programming

```
int uniquePaths(int m, int n) {
```

```
    int dp[n];

    for (int i = 0; i < n; i++)
        dp[i] = 1;

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[j] += dp[j - 1];
        }
    }

    return dp[n - 1];
}

int main() {
    printf("Output: %d\n", uniquePaths(7, 3));
    printf("Output: %d\n", uniquePaths(3, 2));

    return 0;
}
```

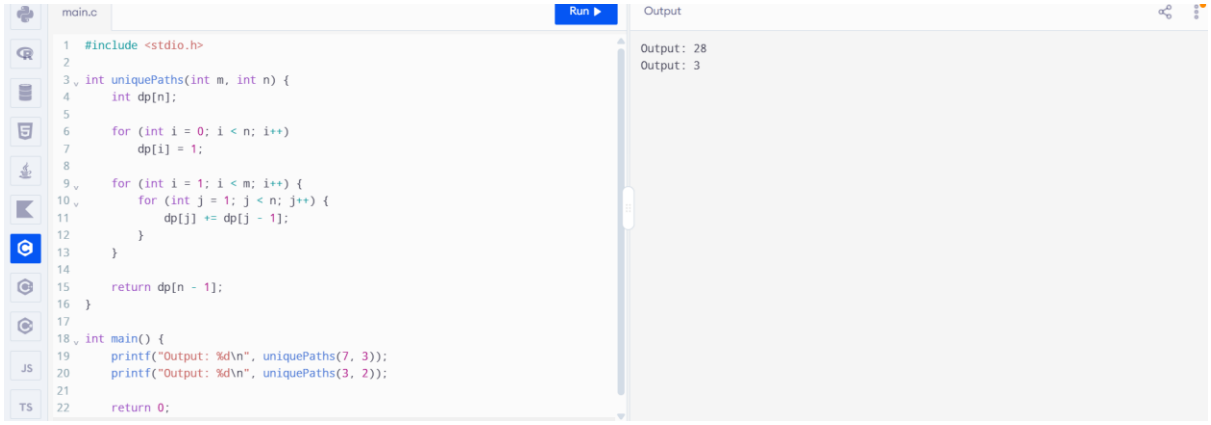
Output

Output: 28

Output: 3

Time Complexity: $O(m \times n)$

Space Complexity: $O(n)$


main.c
Run
Output
#include <stdio.h>
int uniquePaths(int m, int n) {
int dp[n];
for (int i = 0; i < n; i++)
dp[i] = 1;
for (int i = 1; i < m; i++) {
for (int j = 1; j < n; j++) {
dp[j] += dp[j - 1];
}
}
return dp[n - 1];
}
int main() {
printf("Output: %d\n", uniquePaths(7, 3));
printf("Output: %d\n", uniquePaths(3, 2));
return 0;
Output: 28
Output: 3